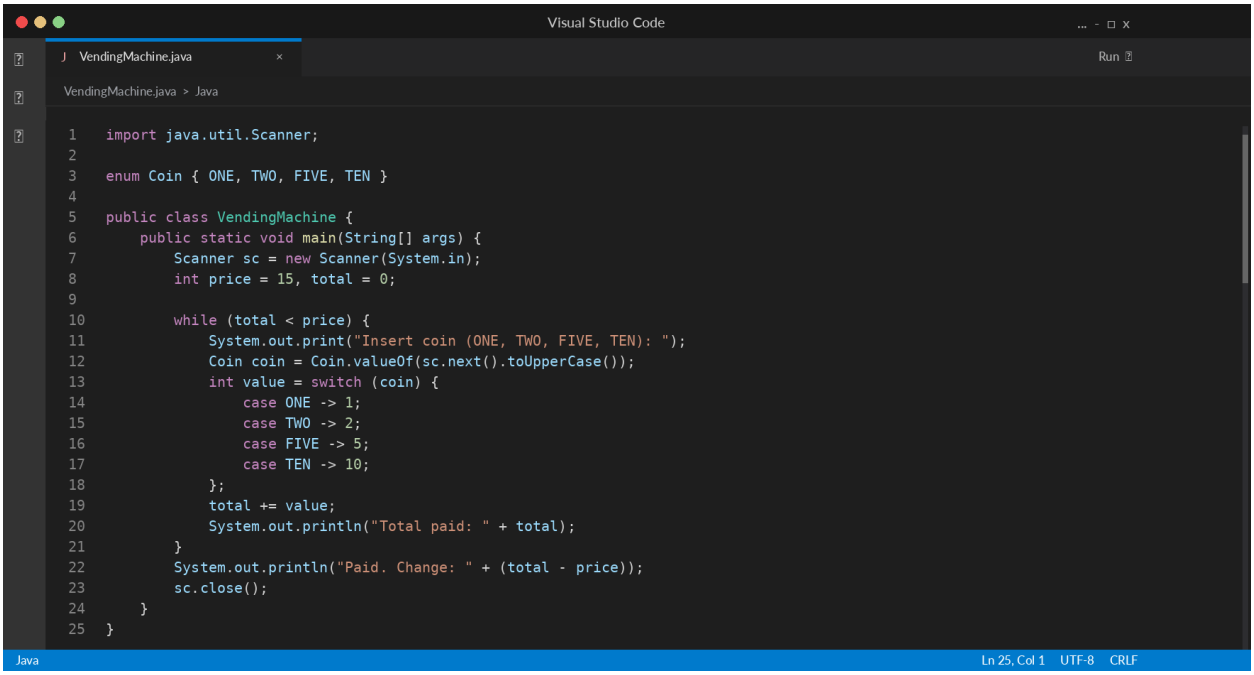
PRACTICAL - 01

Aim: Vending machine (VendingMachine.java)

Code Screenshot:



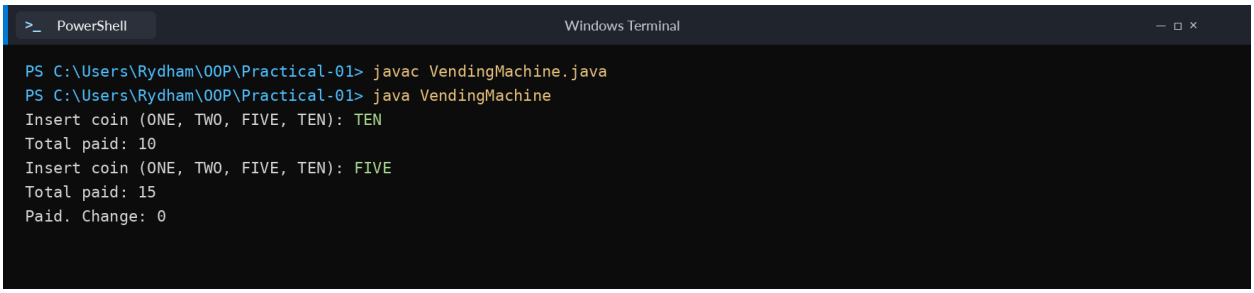

```java
import java.util.Scanner;

enum Coin { ONE, TWO, FIVE, TEN }

public class VendingMachine {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int price = 15, total = 0;

        while (total < price) {
            System.out.print("Insert coin (ONE, TWO, FIVE, TEN): ");
            Coin coin = Coin.valueOf(sc.next().toUpperCase());
            int value = switch (coin) {
                case ONE -> 1;
                case TWO -> 2;
                case FIVE -> 5;
                case TEN -> 10;
            };
            total += value;
            System.out.println("Total paid: " + total);
        }
        System.out.println("Paid. Change: " + (total - price));
        sc.close();
    }
}
```

Output Screenshot:

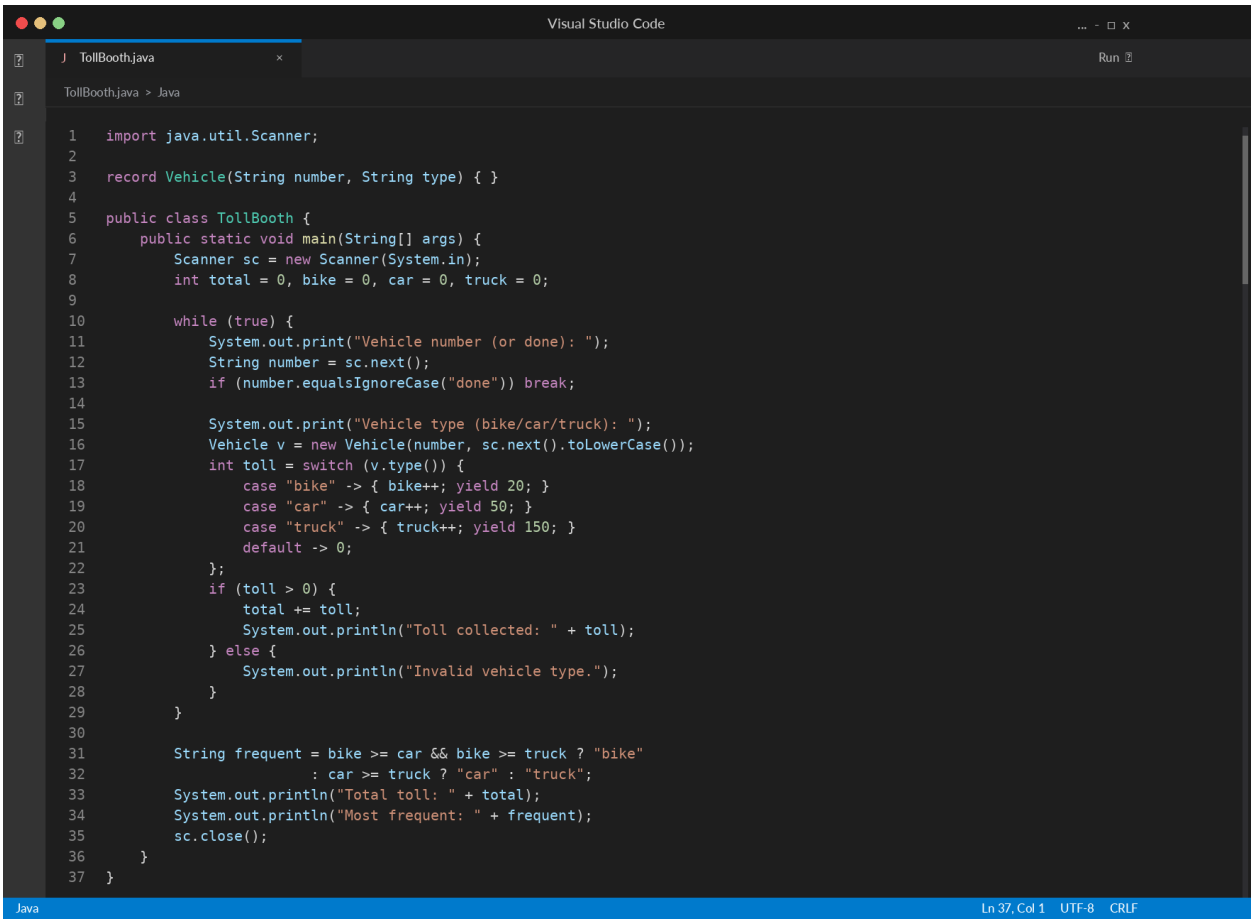


Learning Outcome:

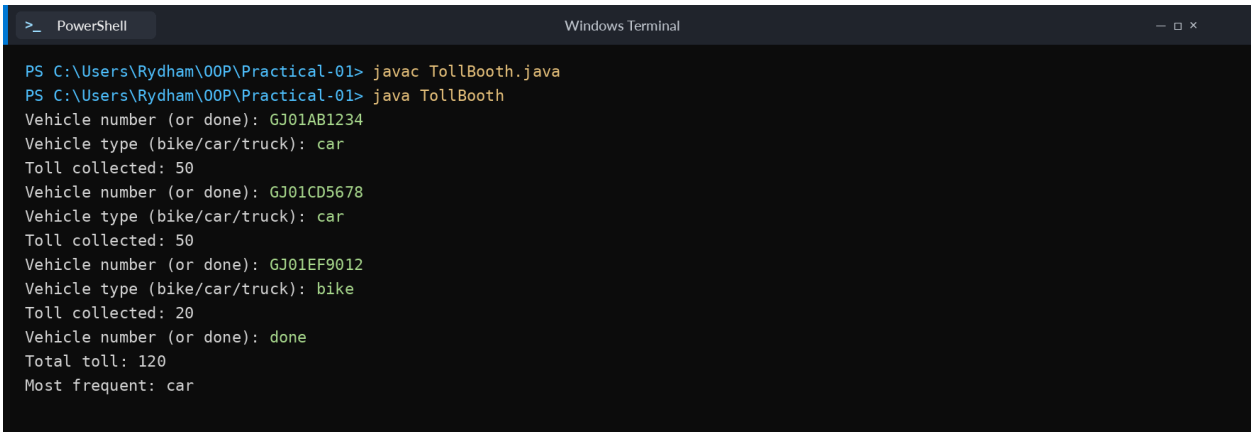
- Learned how to define and utilize Java enum types for fixed sets of constants.
- Understood modern switch expressions with arrow syntax and return values.
- Implemented accumulator logic for iterative coin transaction management.
- Handled console input parsing and enum constant resolution via valueOf().
- Computed exact monetary change based on user input.

Aim: Toll booth (TollBooth.java)

Code Screenshot:



Output Screenshot:

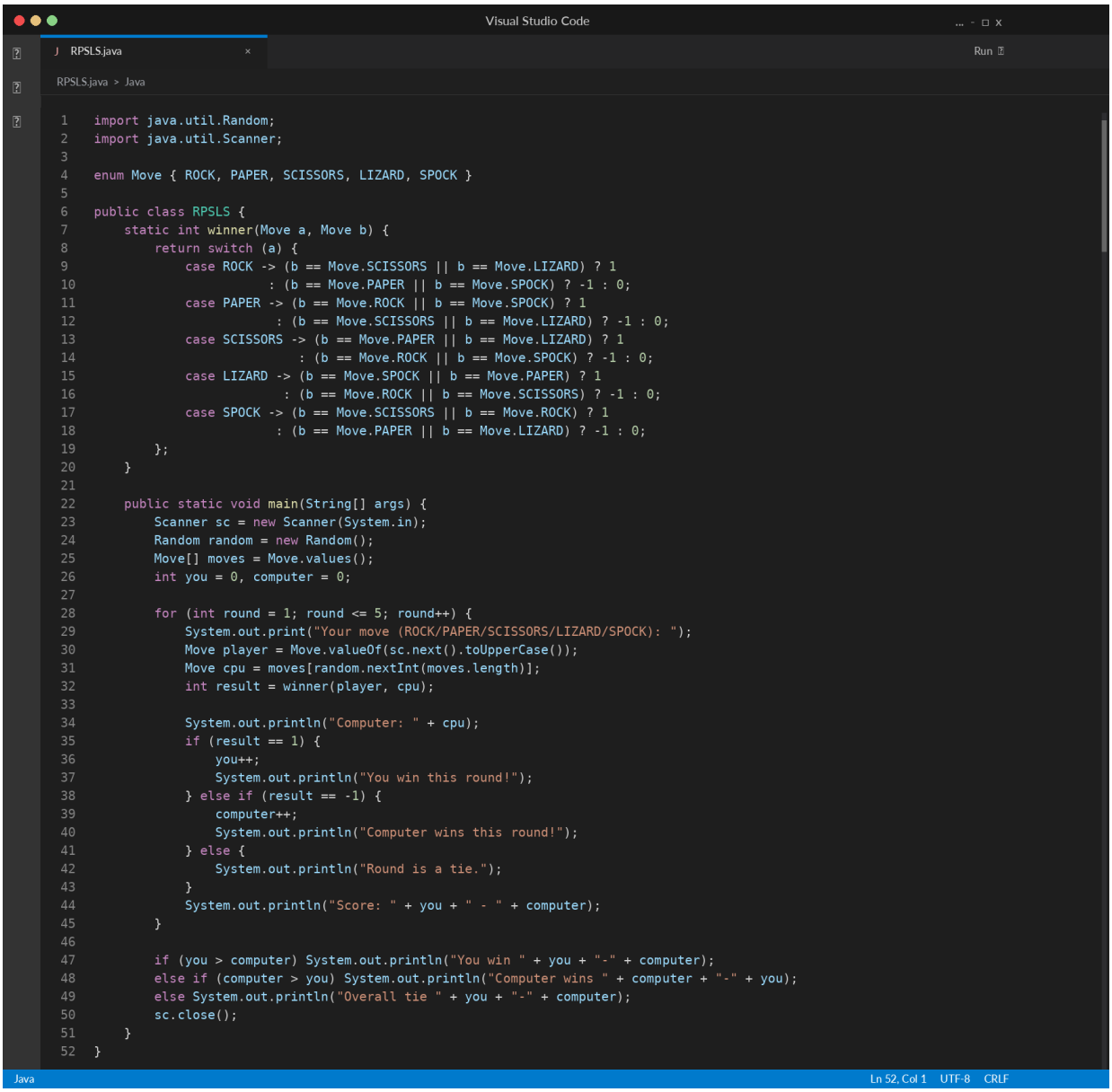


Learning Outcome:

- Mastered Java record types for concise, immutable data holding.
- Used switch expressions with multi-statement blocks and the yield statement.
- Implemented frequency tracking counters across heterogeneous categories.
- Applied loop control flows with sentinel termination conditions.
- Gained hands-on experience in structured object-oriented problem solving.

Aim: Rock-Paper-Scissors-Lizard-Spock (RPSLS.java)

Code Screenshot:




```java
import java.util.Random;
import java.util.Scanner;

enum Move { ROCK, PAPER, SCISSORS, LIZARD, SPOCK }

public class RPSLS {
    static int winner(Move a, Move b) {
        return switch (a) {
            case ROCK -> (b == Move.SCISSORS || b == Move.LIZARD) ? 1
                       : (b == Move.PAPER || b == Move.SPOCK) ? -1 : 0;
            case PAPER -> (b == Move.ROCK || b == Move.SPOCK) ? 1
                        : (b == Move.SCISSORS || b == Move.LIZARD) ? -1 : 0;
            case SCISSORS -> (b == Move.PAPER || b == Move.LIZARD) ? 1
                           : (b == Move.ROCK || b == Move.SPOCK) ? -1 : 0;
            case LIZARD -> (b == Move.SPOCK || b == Move.PAPER) ? 1
                         : (b == Move.ROCK || b == Move.SCISSORS) ? -1 : 0;
            case SPOCK -> (b == Move.SCISSORS || b == Move.ROCK) ? 1
                        : (b == Move.PAPER || b == Move.LIZARD) ? -1 : 0;
        };
    }

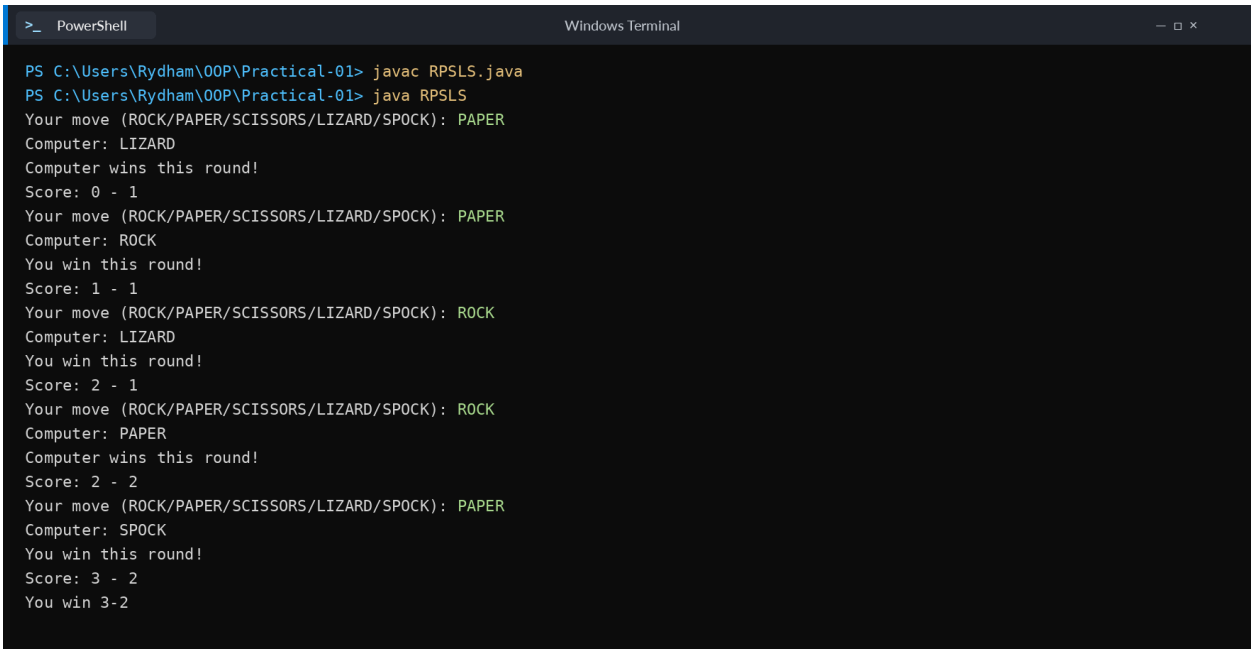
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Random random = new Random();
        Move[] moves = Move.values();
        int you = 0, computer = 0;

        for (int round = 1; round <= 5; round++) {
            System.out.print("Your move (ROCK/PAPER/SCISSORS/LIZARD/SPOCK): ");
            Move player = Move.valueOf(sc.next().toUpperCase());
            Move cpu = moves[random.nextInt(moves.length)];
            int result = winner(player, cpu);

            System.out.println("Computer: " + cpu);
            if (result == 1) {
                you++;
                System.out.println("You win this round!");
            } else if (result == -1) {
                computer++;
                System.out.println("Computer wins this round!");
            } else {
                System.out.println("Round is a tie.");
            }
            System.out.println("Score: " + you + " - " + computer);
        }

        if (you > computer) System.out.println("You win " + you + "-" + computer);
        else if (computer > you) System.out.println("Computer wins " + computer + "-" + you);
        else System.out.println("Overall tie " + you + "-" + computer);
        sc.close();
    }
}
```

Output Screenshot:



Learning Outcome:

- Defined Java enum constants to represent the five game moves.
- Used a switch expression to determine the winner for each pair of moves.
- Implemented randomized computer selections and five-round gameplay.
- Maintained separate scores for the player and computer, including ties.
- Applied conditional logic to display the overall match winner.